Design and Implementation of a Parameterized Cache Controller with FSM-based Memory Management

Piyush Maru (23BEC146) and Tejas Raichura (23BEC154)
Department of Electronics and Communication Engineering
Nirma University
Ahmedabad, India

Abstract

This paper discusses the design and implementation of an FSM-based parameterized cache controller. Cache memory serves as an intermediate layer between a fast processor and slower main (backing) memory, playing a crucial role in improving overall system performance [1].

The proposed cache controller is designed to be configurable for different cache sizes, block sizes, associativity levels, and replacement policies, providing flexibility to adapt to various system requirements [2]. The controller architecture is implemented using a Finite State Machine (FSM) model, which provides a systematic control mechanism for cache memory operations such as read, write, cache hit detection, cache miss handling, cache write-back, and memory refill cycles [2]. The controller generates control signals efficiently and ensures proper timing between the processor, cache, and main memory.

Simulation results demonstrate correct functionality for a range of cache access scenarios, including cache hits, compulsory misses, conflict misses, and write-back operations [3]. The parameterized design enables scalable and reusable controller architectures suitable for modern embedded and processor-based systems [2]. Performance analysis shows improved memory access efficiency compared to systems without cache, while maintaining manageable design complexity [1].

Keywords: Cache Controller, Finite State Machine (FSM), Parameterized Design, Memory Hierarchy, Cache Replacement Policy

1 Introduction

The speed and performance of computer processors have continued to improve over time; however, a significant gap remains between the speed of main memory (RAM) and CPU. This gap results in increased memory latency, negatively impacting system performance [1]. Cache memory serves as an intermediate storage layer between the CPU and main memory, reducing the average memory access time [1].

A cache controller manages data transfers between the CPU and cache memory, and between cache memory and main memory [2]. It determines whether a memory access results in a cache hit or miss and takes appropriate actions, such as fetching data from main memory, writing to cache, or refilling memory [2].

Finite State Machine (FSM) design is widely used for cache controllers due to its predictable, structured, and deterministic nature [3]. FSM-based cache controllers simplify control logic, reduce design complexity, and ensure reliable performance across different memory operations [3].

The parameterized cache controller design allows runtime configuration of cache size, block size, associativity, and replacement policy, enhancing scalability and reusability [4]. This enables integration into various embedded and processor-based systems without modifying the core FSM logic [4].

2 Literature Survey / State of the Art

Cache memory is critical in modern computers because it allows faster data access, improving overall system performance [1]. Early cache designs were based on direct-mapped caches, which were simple and fast but suffered from conflict misses when multiple data blocks mapped to the same cache line [1].

To overcome these limitations, set-associative caches were developed [2]. Smith (1982) proposed designs allowing multiple placement options for a block, improving hit rates but increasing complexity [2].

Research has explored various caching algorithms such as Least Recently Used (LRU), First-In-First-Out (FIFO), and random replacement policies to balance performance and simplicity [3]. Cache controller design focuses on:

- Determining cache hits or misses
- Selecting blocks for replacement after misses
- Handling write-back to main memory
- Refilling cache from memory

FSM-based cache controllers provide deterministic behavior, timing correctness, and support a wide range of cache configurations [4]. Parameterized designs allow configurable cache parameters at compile time, increasing scalability and reuse [4].

Recent works, such as Al-Hashimi et al., demonstrated parameterized cache designs improving performance and scalability [3]. Qualcomm's advancements in cache and replacement strategies aim to reduce miss penalties in multi-core systems [4]. However, current controllers still lack fully configurable FSMs for diverse associativity and replacement policies without significant redesign efforts.

3 Limitations of Existing System

- **Limited Configurability:** Many cache controllers have fixed parameters such as cache size, block size, and associativity. Modifying these parameters requires redesign.
- **High Design Complexity:** Associative caches require multiple comparators and complex replacement logic, increasing silicon area and power consumption.
- **Inefficient Replacement Policies:** Full LRU is expensive for high associativity; approximate methods may degrade performance.
- **Increased Miss Penalty:** Cache misses stall the processor; write-back caches further increase latency.
- **Limited Multi-Core Scalability:** Traditional controllers are designed for single-core systems, complicating cache coherence.
- **FSM State Explosion:** New features like write-through/write-back, burst transfers, and prefetching increase FSM states, making verification complex.
- **High Power Consumption:** Frequent tag comparisons and memory accesses increase dynamic power usage.
- **Poor Adaptability:** Fixed cache configurations may perform poorly for workloads with irregular memory access patterns.

4 Proposed Solution / Methodology

The objective of this work is to design and implement a cache controller using a Finite State Machine (FSM) approach [2]. The controller manages communication between the processor, cache memory, and main memory while handling read and write operations efficiently [1].

The FSM-based design provides a structured control mechanism in which each state represents a specific cache operation [2]. The controller transitions between states based on processor requests, hit/miss conditions, and memory response signals.

The primary states of the FSM include:

1. **Idle State:** The controller waits for a read or write request from the processor.
2. **Tag Comparison State:** The requested address tag is compared with stored cache tags to determine hit or miss [1].
3. **Read/Write Hit State:** If a cache hit occurs, data is either read from or written to the cache depending on the operation [1].

4. **Miss Handling State:** In case of a cache miss, the controller initiates access to main memory [1].
5. **Write-Back State (for write-back policy):** If the selected cache block is dirty, it is written back to main memory before replacement [3].
6. **Allocate/Refill State:** The requested memory block is fetched from main memory and stored in the cache [1].

The controller is parameterized to allow configuration of cache size, block size, and associativity at design time, improving adaptability for different systems [4]. FSM-based design ensures deterministic operation, simplified verification, and clear control signal generation [2]. Implementation is done in HDL, and simulations validate cache functionality under different access scenarios.

4.1 System Architecture

The proposed cache system acts as an intermediate layer between the CPU and main memory. It consists of the following components:

- Address decoding unit
- Direct-mapped cache memory
- FSM-based cache controller

Example : Suppose CPU address is 16-bit,then it is divided as follows:

- Tag: 9 bits
- Index: 3 bits
- Offset: 4 bits

The index selects one of the 8 cache lines. Each cache line stores:

- 128-bit data block
- 9-bit tag
- Valid bit

The FSM-based controller compares the input tag with stored tags and checks the valid bit to determine a hit or miss.

The system follows a **write-back, write-allocate policy**:

- **Read Hit:** Data is read directly from the cache memory.

- **Read Miss:** The required block is fetched from main memory and allocated to the cache.
- **Write Hit:** Data is written only to the cache, and the corresponding cache line is marked as dirty.
- **Write Miss:** The cache controller allocates the required block from main memory into the cache before performing the write operation (write-allocate policy).
- **Cache Replacement:** If a dirty cache block is replaced, it is written back to main memory before the new block is loaded.

This architecture reduces memory access latency while maintaining data consistency between cache and main memory.

5 Block Diagram

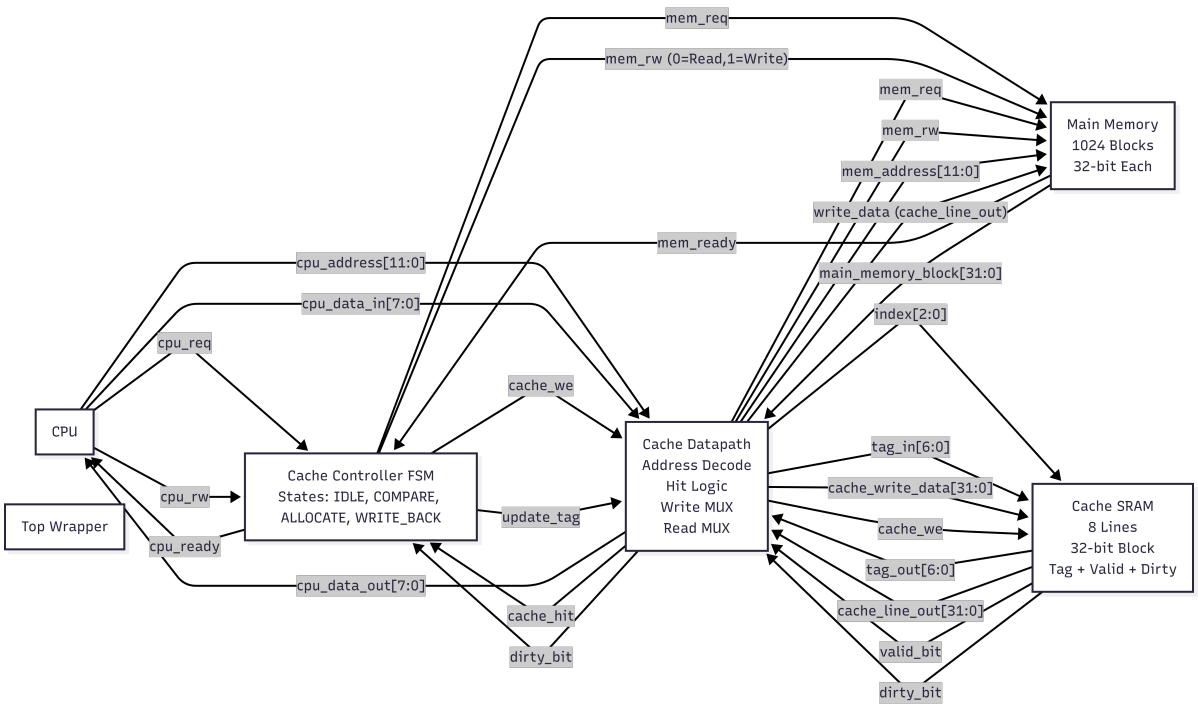

Figure 1: Block Diagram of Proposed System

The system architecture consists of a CPU interface, a cache controller FSM, a cache datapath, cache SRAM, and main memory. The CPU sends read/write requests and addresses to the cache datapath. The FSM manages cache operations, handling hits, misses, allocation, and write-back. The cache SRAM stores data, tags, valid, and dirty bits, while main memory provides block transfers during cache misses.

6 Flow Chart

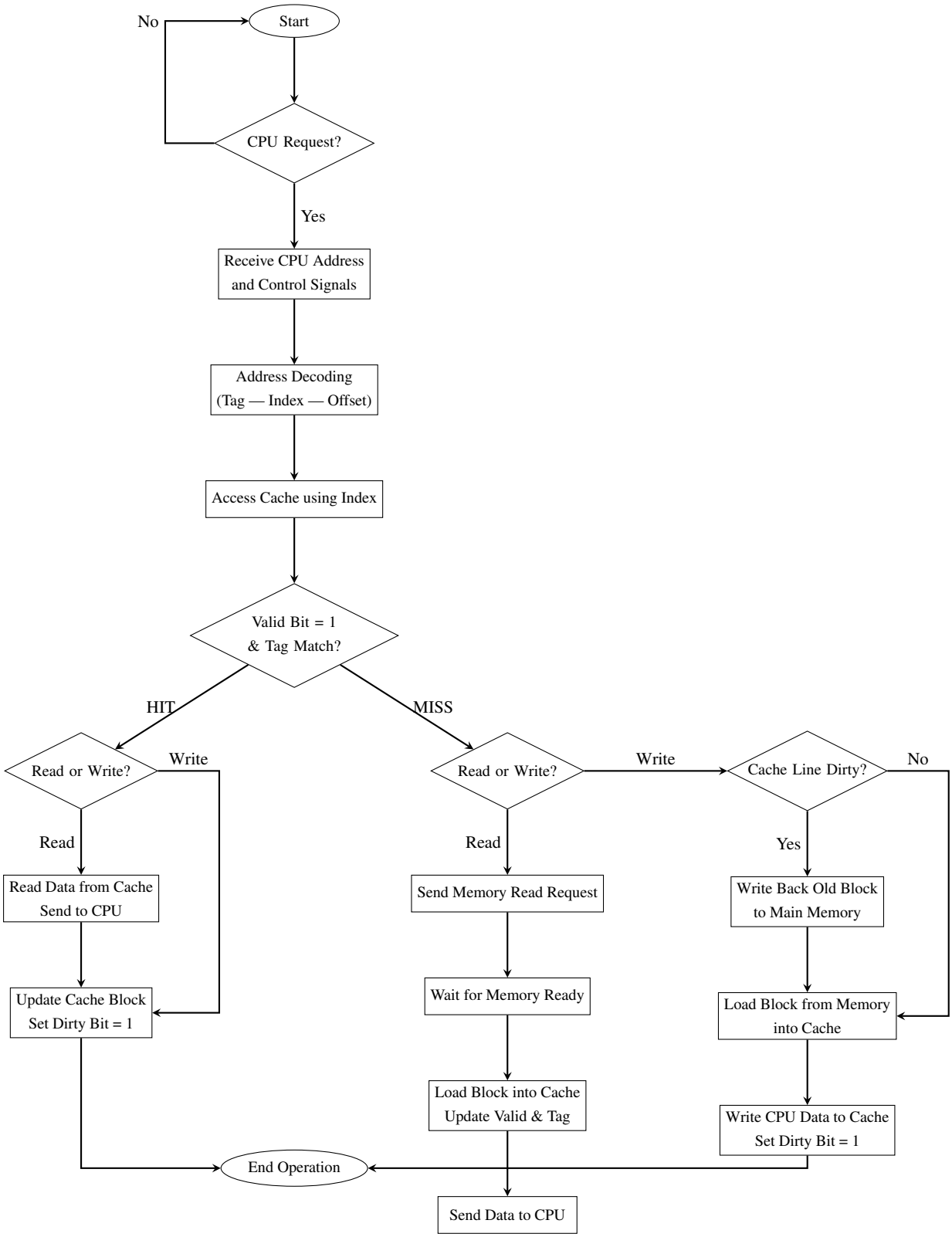

Figure 2: Flowchart of Direct-Mapped Cache Controller using Write-Back and Write-Allocate Policy

7 Analysis of the Obtained Results

The simulation results validate the correct functioning of the proposed cache controller under different memory access scenarios. The following figures illustrate the observed waveform outputs during simulation.

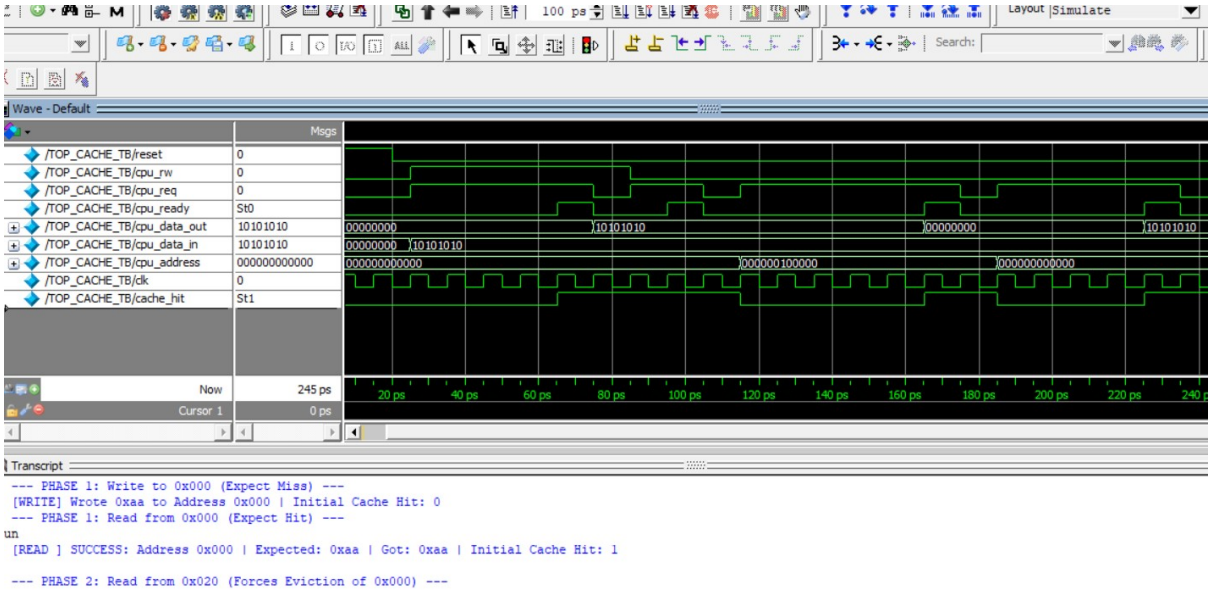

Figure 3: Modelsim Waveform Simulation

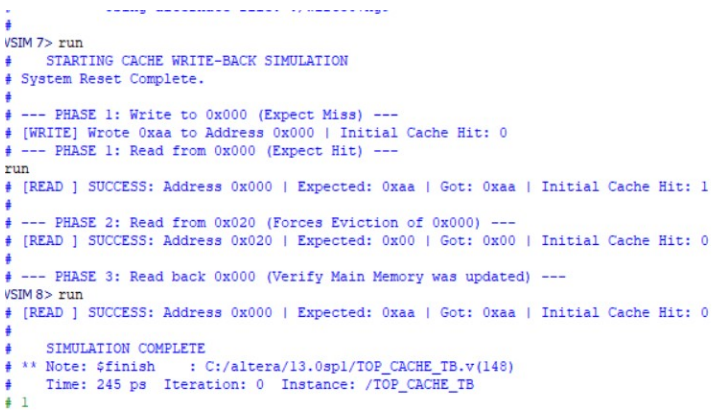

Figure 4: Data Printed

Figure 4 illustrates the cache miss condition. When the requested block is not present in the cache, the controller generates a memory read request to fetch the block from main memory. After the memory responds with the required data, the block is loaded into the cache and the CPU request is completed. This behavior confirms correct implementation of the write-allocate policy.

8 Additional Relevant Information

8.1 Parameterized Design Strategy

The proposed cache controller is developed using configurable parameters such as cache size, block size, address width, number of cache lines, tag width, index width, and offset width [2]. These parameters are defined using generics (in VHDL) or parameters (in Verilog), enabling flexible configuration of the cache architecture without modifying the core FSM structure [4].

This approach improves scalability, reusability, and adaptability across different processor architectures [4].

8.2 FSM-Based Memory Management

The controller utilizes a Finite State Machine (FSM) to manage memory transactions between the CPU, cache, and main memory. The FSM is responsible for:

- Handling read and write requests
- Performing tag comparison
- Detecting hit and miss conditions
- Generating memory control signals
- Updating cache blocks during miss conditions

The FSM ensures deterministic control flow and proper synchronization with main memory latency.

8.3 Control and Timing Signals

The cache controller generates essential control signals, including:

- Cache_Read
- Cache_Write
- Mem_Read
- Mem_Write
- Hit and Miss indicators
- Ready/Wait signals

During a miss condition, the FSM introduces wait states until the required data is fetched from main memory.

8.4 Scalability and Configurability

The parameterized architecture enables:

- Modification of cache size without redesign
- Adaptation to different processor address widths
- Extension to alternative cache organizations

Only parameter values need adjustment, while the FSM logic remains unchanged.

8.5 Performance Considerations

System performance is influenced by cache hit rate, miss penalty, block size selection, and memory access latency. The parameterized design allows experimentation with various configurations to achieve optimal performance for different workloads.

8.6 Hardware Optimization

Hardware complexity is minimized by:

- Using direct mapping (single tag comparator)
- Separating tag and data arrays
- Reducing combinational logic in FSM transitions

This makes the design suitable for FPGA and embedded implementations.

8.7 Limitations and Future Scope

Limitations:

- Higher conflict miss rate in direct-mapped architecture
- No runtime dynamic reconfiguration
- Basic write-through policy implementation

Future Enhancements:

- Support for write-back policy
- Multi-level cache hierarchy integration
- Extension to set-associative structures
- Adaptive cache parameter tuning

9 Conclusion

The design of a Parameterized Direct-Mapped Cache Controller using FSM-based Memory Management shows how we can cut down on memory access delays with smart hardware controls

By adjusting key design elements like cache size, block size, and address width, the architecture gains scalability and adaptability, enabling the same controller setup to be used across various system setups without needing a complete overhaul This flexible and adaptable method mirrors today's trends in building hardware

The FSM- based control logic enables systematic handling of cache operations including hit detection, miss handling, block refill, and CPU stall management. The finite state machine guaranties consistent and predictable actions when dealing with memory stuff, making things more reliable and easier to check.

References

[1] David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*. Morgan Kaufmann, 2017.

[2] John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 2019.

[3] NPTEL Course, *Computer Architecture*. Available at: https://nptel.ac.in

[4] MIT OpenCourseWare, *Computer System Architecture*. Available at: https://ocw.mit.edu

A Verilog Implementation of Cache Controller

Listing 1: Parameterized Cache Controller Verilog Code

```

module top_wraper(
    input wire clk,
    input wire reset,

    input wire cpu_req,
    input wire cpu_rw,
    input wire [11:0] cpu_address,
    input wire [7:0] cpu_data_in,

    output wire [7:0] cpu_data_out,
    output wire cpu_ready,
    output wire cache_hit
);

    wire fsm_cache_we;
    wire fsm_update_tag;
    wire fsm_mem_req;
    wire fsm_mem_rw;
    wire mem_ready_signal;
    wire datapath_hit;
    wire mem_to_cache_mux;

    wire real_dirty_bit_signal;

    assign cache_hit = datapath_hit;

    assign mem_to_cache_mux = (fsm_mem_req && !fsm_mem_rw);

    cache_controller_fsm FSM (
        .clk(clk),
        .reset(reset),
        .cpu_req(cpu_req),
        .cpu_rw(cpu_rw),
        .cpu_ready(cpu_ready),
        .cache_hit(datapath_hit),
        .cache_dirty(real_dirty_bit_signal),
        .cache_we(fsm_cache_we),
        .update_tag(fsm_update_tag),
        .mem_ready(mem_ready_signal),
        .mem_req(fsm_mem_req),
        .mem_rw(fsm_mem_rw)
    );
```

```
    cache_datapath DATAPATH (
        .clock(clk),
        .reset(reset),
        .cpu_address(cpu_address),
        .cpu_data_in(cpu_data_in),
        .cpu_data_out(cpu_data_out),
        .hit(datapath_hit),
        .dirty_bit(real_dirty_bit_signal),
        .cache_we(fsm_cache_we),
        .mem_to_cache(mem_to_cache_mux),
        .mem_req(fsm_mem_req),
        .mem_rw(fsm_mem_rw),
        .mem_ready(mem_ready_signal)
    );

endmodule


module cache_datapath (
    input clock,
    input reset,

    input [11:0] cpu_address,
    input [7:0] cpu_data_in,
    output [7:0] cpu_data_out,
    output hit,

    output dirty_bit,

    input cache_we,
    input mem_to_cache,
    input mem_req,
    input mem_rw,

    output mem_ready
);

    wire [1:0] offset;
    wire [2:0] index;
    wire [6:0] tag;

    wire [6:0] tag_out;
    wire valid_bit;
    wire [31:0] cache_line_out;
    wire [31:0] main_memory_block;
    wire [31:0] cache_write_data;
```

```
    assign offset = cpu_address[1:0];
    assign index  = cpu_address[4:2];
    assign tag    = cpu_address[11:5];

    assign hit = (tag == tag_out) && valid_bit;

    assign cpu_data_out = (offset == 2'b00) ? cache_line_out[7:0]   :
                          (offset == 2'b01) ? cache_line_out[15:8]  :
                          (offset == 2'b10) ? cache_line_out[23:16] :
                                              cache_line_out[31:24] ;

    assign cache_write_data = mem_to_cache ? main_memory_block :
                              (offset == 2'b00) ? {cache_line_out[31:8], cpu_data_in} :
                              (offset == 2'b01) ? {cache_line_out[31:16], cpu_data_in, cache_line_out[7:0]} :
                              (offset == 2'b10) ? {cache_line_out[31:24], cpu_data_in, cache_line_out[15:0]} :
                                                  {cpu_data_in, cache_line_out[23:0]};

    wire [11:0] mem_address;

    assign mem_address = mem_rw ? {tag_out, index, 2'b00} : {cpu_address[11:2], 2'b00};

    cache_sram CSRAM (
        .clock(clock),
        .reset(reset),
        .index(index),
        .we(cache_we),
        .mem_to_cache(mem_to_cache),
        .tag_in(tag),
        .data_in(cache_write_data),
        .tag_out(tag_out),
        .data_out(cache_line_out),
        .valid_out(valid_bit),
        .dirty_out(dirty_bit)
    );

    main_memory MM (
        .clock(clock),
        .mem_req(mem_req),
        .mem_rw(mem_rw),
        .address(mem_address),
        .data_in(cache_line_out),
```

```
        .data_out(main_memory_block),
        .mem_ready(mem_ready)
    );

endmodule


module cache_sram(
    input clock,
    input reset,
    input [2:0] index,
    input we,
    input mem_to_cache,
    input [6:0] tag_in,
    input [31:0] data_in,
    output [6:0] tag_out,
    output [31:0] data_out,
    output valid_out,
    output dirty_out
);

    reg [31:0] data_array [0:7];
    reg [6:0]  tag_array  [0:7];
    reg        valid_array[0:7];
    reg        dirty_array[0:7];

    integer i;

    always @(posedge clock) begin
        if (reset) begin
            for(i = 0; i < 8; i = i + 1) begin
                data_array[i]  <= 32'b0;
                tag_array[i]   <= 7'b0;
                valid_array[i] <= 1'b0;
                dirty_array[i] <= 1'b0;
            end
        end else if (we) begin
            data_array[index]  <= data_in;
            tag_array[index]   <= tag_in;
            valid_array[index] <= 1'b1;
            dirty_array[index] <= !mem_to_cache;
        end
    end

    assign data_out  = data_array[index];
    assign tag_out   = tag_array[index];
    assign valid_out = valid_array[index];
```

```
    assign dirty_out = dirty_array[index];

endmodule


module main_memory(
    input clock,
    input mem_req,
    input mem_rw,
    input [11:0] address,
    input [31:0] data_in,
    output reg [31:0] data_out,
    output reg mem_ready
);

    reg [31:0] memory [0:1023];
    wire [9:0] block_addr;

    assign block_addr = address[11:2];

    integer i;
    initial begin
        for(i = 0; i < 1024; i = i + 1) begin
            memory[i] = 32'h00000000;
        end
    end

    always @(posedge clock) begin
        mem_ready <= 1'b0;

        if (mem_req) begin
            if (mem_rw == 1'b0) begin
                data_out <= memory[block_addr];
            end else begin
                memory[block_addr] <= data_in;
            end
            mem_ready <= 1'b1;
        end
    end

endmodule


module cache_controller_fsm (
    input wire clk,
    input wire reset,

```

```
    input wire cpu_req,
    input wire cpu_rw,
    output reg cpu_ready,

    input wire cache_hit,
    input wire cache_dirty,
    output reg cache_we,
    output reg update_tag,

    input wire mem_ready,
    output reg mem_req,
    output reg mem_rw
);

    localparam [1:0]
        IDLE       = 2'b00,
        COMPARE    = 2'b01,
        ALLOCATE   = 2'b10,
        WRITE_BACK = 2'b11;

    reg [1:0] current_state, next_state;

    always @(posedge clk or posedge reset) begin
        if (reset) current_state <= IDLE;
        else       current_state <= next_state;
    end

    always @(*) begin
        next_state = current_state;
        case (current_state)
            IDLE: if (cpu_req) next_state = COMPARE;
            COMPARE: begin
                if (cache_hit)           next_state = IDLE;
                else if (cache_dirty)    next_state = WRITE_BACK;
                else                     next_state = ALLOCATE;
            end
            ALLOCATE:   if (mem_ready) next_state = COMPARE;
            WRITE_BACK: if (mem_ready) next_state = ALLOCATE;
            default: next_state = IDLE;
        endcase
    end

    always @(*) begin
        cpu_ready  = 1'b0;
        cache_we   = 1'b0;
        update_tag = 1'b0;
        mem_req    = 1'b0;
```

```
        mem_rw     = 1'b0;

        case (current_state)
            COMPARE: begin
                if (cache_hit) begin
                    cpu_ready = 1'b1;
                    if (cpu_rw) begin
                        cache_we = 1'b1;
                        update_tag = 1'b1;
                    end
                end
            end
            ALLOCATE: begin
                mem_req = 1'b1;
                mem_rw  = 1'b0;
                if (mem_ready) begin
                    cache_we   = 1'b1;
                    update_tag = 1'b1;
                end
            end
            WRITE_BACK: begin
                mem_req = 1'b1;
                mem_rw  = 1'b1;
            end
        endcase
    end
endmodule
```